Пензенский государственный университет
Факультет вычислительной техники
Кафедра «Вычислительная техника»

Обработка исключительных ситуаций

Отчет о лабораторной работе № 3
по дисциплине «Разработка кроссплатформенных приложений»
Вариант 1

Выполнили: студенты гр. 23ВОЭ1
Чиндин И.Д.
Снигирев А.А.
Проверили: к.т.н., доцент
Юрова О.В.
к.т.н., доцент
Карамышева Н.С.

2026

1 Цель работы

Изучить механизм обработки исключительных ситуаций.

2 Формулировка задачи

Модифицировать приложение из предыдущей лабораторной работы, реализовав проверку вводимых данных с использованием механизма исключений. Необходимо создать свой класс, унаследованный от класса Exception, и генерировать исключение, если возникает попытка создать экземпляр класса RecIntegral со значениями, не являющимися числами в диапазоне от 0,000001 до 1000000. В качестве обработки исключения необходимо выводить диалог, содержащий предупреждение о некорректности введенных данных.

3 Ход работы

Чтобы модифицировать программу и реализовать класс, унаследованный от класса Exception, был создан класс InvalidRec.

Класс InvalidRec представляет собой пользовательское исключение, предназначенное для обработки недопустимых параметров интегрирования в программе. Он объявлен как наследник базового класса Exception.

Конструктор класса InvalidRec принимает строковый параметр message и передаёт его в конструктор класса Exception с помощью вызова super(message). При возникновении ошибки можно перехватить это исключение в блоке try-catch и вывести пользователю сообщение. Листинг приведен в приложении А.

Так же необходимо изменить конструктор класса RecIntegral. В обоих конструкторах класса выполняется проверка всех переданных значений с помощью метода isValid(). Если хотя бы один параметр выходит за допустимые

границы от 0.000001 до 1 000 000 генерируется пользовательское исключение InvalidRec с сообщением о ошибке. Первый конструктор инициализирует пределы и шаг, устанавливая результат равным нулю, а второй позволяет задать и уже вычисленное значение результата.

Метод isValid() возвращает логическое значение, указывающее, находится ли число в допустимом диапазоне.

Так же необходимо изменить методы обработчиков событий и добавить к кнопке «Добавить» обработчик исключений. Обработчик кнопки «Добавить» обеспечивает ввод новых данных в таблицу. При нажатии выполняется проверка: если хотя бы одно из текстовых полей пустое, пользователю выводится сообщение «Заполните все поля!». В случае ошибки формата отображается сообщение «Введите корректные числовые значения!». Если данные корректны, создаётся объект RecIntegral, который проходит проверку допустимости значений в своём конструкторе. Затем новая строка с параметрами интегрирования и нулевым результатом добавляется в таблицу, а объект сохраняется в коллекции arrInteg. После добавления поля ввода очищаются.

Обработчик кнопки «Рассчитать» выполняет вычисление значения интеграла для выбранной строки таблицы. Сначала определяется индекс выделенной строки, и если строка не выбрана, пользователю выводится сообщение «Выберите строку!». Если строка выбрана, из неё извлекаются значения нижнего и верхнего пределов и шага интегрирования. На основе этих данных создаётся объект RecIntegral, после чего вызывается метод CalcIntegral, вычисляющий приближённое значение интеграла методом трапеций. Полученный результат сохраняется в объекте и записывается в четвёртый столбец таблицы. Затем обновлённый объект заменяет старый в коллекции arrInteg, что обеспечивает согласованность данных. Листинг приведён в приложении А.

Обработчик кнопки «Удалить» позволяет удалить выбранную строку таблицы. Он получает индекс выделенной строки, и если строка не выбрана, выводит предупреждение «Enter string». Если строка выделена, она удаляется из таблицы методом removeRow(rowNum), а соответствующий объект удаляется из коллекции arrInteg.

Обработчик кнопки «Очистить» полностью очищает таблицу, устанавливая количество строк равным нулю. При этом коллекция arrInteg не очищается, что позволяет при необходимости восстановить данные обратно в таблицу. Листинг приведён в приложении А.

Обработчик кнопки «Заполнить» выполняет обратную операцию — он восстанавливает содержимое таблицы из коллекции arrInteg. При нажатии таблица предварительно очищается, после чего для каждого объекта интеграла добавляется строка, содержащая его параметры и ранее вычисленный результат. Это позволяет пользователю повторно загрузить данные после очистки таблицы.

Результаты выполнения программы приведены в приложении Б (рисунок Б.1, Б.2).

Вывод

В ходе выполнения лабораторной работы было модифицировано приложение из предыдущей лабораторной работы, реализована проверка вводимых данных с использованием механизма исключений. Создать свой класс, унаследованный от класса Exception, и сгенерировано исключение, если возникает попытка создать экземпляр класса RecIntegral со значениями, не являющимися числами в диапазоне от 0,000001 до 1000000. В качестве обработки исключения необходимо выводится диалог, содержащий предупреждение о некорректности введенных данных.

ТЕКСТ ПРОГРАММЫ

Приложение А

(обязательное)

Основная программа:

```
package lab1.pkg1;

import  javax.swing.table.DefaultTableModel;
import  javax.swing.JOptionPane;
import java.util.ArrayList;



/**
 *
 * @author student
 */
public class NewJFrame extends javax.swing.JFrame {
ArrayList<RecIntegral> arrInteg = new ArrayList<>();
    /**
     * Creates new form NewJFrame
     */
    public NewJFrame() {
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        jFrame1 = new javax.swing.JFrame();
```

```
jPanel1 = new javax.swing.JPanel();
jScrollPane1 = new javax.swing.JScrollPane();
jTable1 = new javax.swing.JTable();
jButtonAdd = new javax.swing.JButton();
jButtonDelete = new javax.swing.JButton();
jButtonResult = new javax.swing.JButton();
jTextFieldLow = new javax.swing.JTextField();
jTextFieldUp = new javax.swing.JTextField();
jTextFieldStep = new javax.swing.JTextField();
jLabel1 = new javax.swing.JLabel();
jLabel2 = new javax.swing.JLabel();
jLabel3 = new javax.swing.JLabel();
jLabel4 = new javax.swing.JLabel();
AddColection = new javax.swing.JButton();
DeleteAll = new javax.swing.JButton();

javax.swing.GroupLayout jFrame1Layout = new javax.swing.GroupLayout(jFrame1.getContentPane());
jFrame1.getContentPane().setLayout(jFrame1Layout);
jFrame1Layout.setHorizontalGroup(
    jFrame1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGap(0, 400, Short.MAX_VALUE)
);
jFrame1Layout.setVerticalGroup(
    jFrame1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGap(0, 300, Short.MAX_VALUE)
);

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
setBackground(new java.awt.Color(255, 153, 0));
setBounds(new java.awt.Rectangle(0, 0, 10, 10));
setCursor(new java.awt.Cursor(java.awt.Cursor.DEFAULT_CURSOR));
```

```
setForeground(new java.awt.Color(255, 255, 255));
addComponentListener(new java.awt.event.ComponentAdapter() {
    public void componentHidden(java.awt.event.ComponentEvent evt) {
        formComponentHidden(evt);
    }
});

jPanel1.setBackground(new java.awt.Color(204, 255, 204));
jPanel1.setForeground(new java.awt.Color(204, 0, 51));

jScrollPane1.setForeground(new java.awt.Color(255, 0, 102));

jTable1.setBackground(new java.awt.Color(153, 153, 255));
jTable1.setBorder(new javax.swing.border.MatteBorder(null));
jTable1.setForeground(new java.awt.Color(255, 0, 51));
jTable1.setModel(new javax.swing.table.DefaultTableModel(
    new Object [][] {

    },
    new String [] {
        "Нижний порог", "Верхний порог", "Шаг", "Результат"
    }
) {
    boolean[] canEdit = new boolean [] {
        true, true, true, false
    };

    public boolean isCellEditable(int rowIndex, int columnIndex) {
        return canEdit [columnIndex];
    }
});
```

```
jScrollPane1.setViewportView(jTable1);

jButtonAdd.setBackground(new java.awt.Color(204, 204, 204));
jButtonAdd.setText("Добавить");
jButtonAdd.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButtonAddActionPerformed(evt);
    }
});

jButtonDelete.setBackground(new java.awt.Color(204, 204, 204));
jButtonDelete.setText("Удалить");
jButtonDelete.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButtonDeleteActionPerformed(evt);
    }
});

jButtonResult.setBackground(new java.awt.Color(204, 204, 204));
jButtonResult.setText("Рассчитать");
jButtonResult.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButtonResultActionPerformed(evt);
    }
});

jLabel1.setText("Нижний порог");

jLabel2.setText("Верхний порог");

jLabel3.setText("Шаг");
```

```
jLabel4.setText("1/x");

AddColection.setBackground(new java.awt.Color(204, 204, 204));
AddColection.setText("Заполнить");
AddColection.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        AddColectionActionPerformed(evt);
    }
});

DeleteAll.setBackground(new java.awt.Color(204, 204, 204));
DeleteAll.setText("Очистить");
DeleteAll.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        DeleteAllActionPerformed(evt);
    }
});

javax.swing.GroupLayout jPanel1Layout = new javax.swing.GroupLayout(jPanel1);
jPanel1.setLayout(jPanel1Layout);
jPanel1Layout.setHorizontalGroup(
    jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addComponent(jScrollPane1, javax.swing.GroupLayout.DEFAULT_SIZE, 539, Short.MAX_VALUE)
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, jPanel1Layout.createSequentialGroup()
        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(AddColection, javax.swing.GroupLayout.Alignment.TRAILING)
            .addComponent(DeleteAll, javax.swing.GroupLayout.Alignment.TRAILING))
        .addGap(18, 18, 18)
        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
                .addComponent(jButtonResult)
                .addComponent(jButtonDelete)
                .addComponent(jButtonAdd))
            .addGap(52, 52, 52))
        .addGroup(jPanel1Layout.createSequentialGroup()
            .addGap(21, 21, 21)
            .addComponent(jLabel4)
            .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(jPanel1Layout.createSequentialGroup()
                .addContainerGap(78, Short.MAX_VALUE)
                .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addComponent(jLabel1)
                    .addComponent(jLabel3)
                    .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
                        .addComponent(jTextFieldUp, javax.swing.GroupLayout.PREFERRED_SIZE, 133,
javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addComponent(jTextFieldStep, javax.swing.GroupLayout.PREFERRED_SIZE, 133,
javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addComponent(jTextFieldLow, javax.swing.GroupLayout.PREFERRED_SIZE, 133,
javax.swing.GroupLayout.PREFERRED_SIZE))
                    .addComponent(jLabel2))
                .addGap(328, 328, 328)))
    );
    jPanel1Layout.setVerticalGroup(
        jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, jPanel1Layout.createSequentialGroup()
            .addContainerGap()
            .addComponent(jLabel4)
            .addGap(63, 63, 63)
            .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(jButtonAdd)
                .addComponent(AddColection))
```

```
            .addGap(26, 26, 26)
            .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(jButtonDelete)
                .addComponent(DeleteAll))
            .addGap(26, 26, 26)
            .addComponent(jButtonResult)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 80, Short.MAX_VALUE)
            .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 222,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(jPanel1Layout.createSequentialGroup()
                .addGap(68, 68, 68)
                .addComponent(jLabel1)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(jTextFieldLow, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                .addComponent(jLabel2)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(jTextFieldUp, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                .addComponent(jLabel3)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(jTextFieldStep, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addContainerGap(284, Short.MAX_VALUE)))
    );

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
            .addComponent(jPanel1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        );
        layout.setVerticalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addComponent(jPanel1, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(0, 12, Short.MAX_VALUE))
        );

        pack();
    }// </editor-fold>

    private void jButtonAddActionPerformed(java.awt.event.ActionEvent evt) {
if (jTextFieldLow.getText().isEmpty() ||
        jTextFieldUp.getText().isEmpty() ||
        jTextFieldStep.getText().isEmpty())
    {
        JOptionPane.showMessageDialog(this, "Заполните все поля!");
        return;
    }

    try {
        double lowLim = Double.parseDouble(jTextFieldLow.getText());
        double upLim = Double.parseDouble(jTextFieldUp.getText());
        double step = Double.parseDouble(jTextFieldStep.getText());

        RecIntegral rec = new RecIntegral(lowLim, upLim, step);

        DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
        tModel.addRow(new Object[]{lowLim, upLim, step, 0.0});
```

```
    arrInteg.add(rec);

    jTextFieldLow.setText("");
    jTextFieldUp.setText("");
    jTextFieldStep.setText("");

} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "Введите корректные числовые значения!");
} catch (InvalidRec ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage());
}
}

private void jButtonResultActionPerformed(java.awt.event.ActionEvent evt) {
        try {
    DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
    int rowNum = jTable1.getSelectedRow();

    if (rowNum == -1) {
        JOptionPane.showMessageDialog(this, "Выберите строку!");
        return;
    }

    double lowLim = Double.parseDouble(tModel.getValueAt(rowNum, 0).toString());
    double upLim = Double.parseDouble(tModel.getValueAt(rowNum, 1).toString());
    double step = Double.parseDouble(tModel.getValueAt(rowNum, 2).toString());

    RecIntegral rec = new RecIntegral(lowLim, upLim, step);
    double result = rec.CalcIntegral(lowLim, upLim, step);
    rec.setResult(result);
```

```
        tModel.setValueAt(result, rowNum, 3);
        arrInteg.set(rowNum, rec);

    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Некорректные данные в таблице!");
    } catch (InvalidRec ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage());
    }



// TODO add your handling code here:
    }

    private void formComponentHidden(java.awt.event.ComponentEvent evt) {
        // TODO add your handling code here:
    }

    private void jButtonDeleteActionPerformed(java.awt.event.ActionEvent evt) {
      DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
      int rowNum = jTable1.getSelectedRow();
        if (rowNum == -1) {
            JOptionPane.showMessageDialog(null, "Enter string");
        }
        else
        {  tModel.removeRow(rowNum);
        arrInteg.remove(rowNum);
        }
    }

    private void DeleteAllActionPerformed(java.awt.event.ActionEvent evt) {
```

```
  DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
tModel.setRowCount(0);
//arrInteg.clear();
}

private void AddColectionActionPerformed(java.awt.event.ActionEvent evt) {
  DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
tModel.setRowCount(0); // очистить таблицу

for (RecIntegral rec : arrInteg) {
   tModel.addRow(new Object[]{
      rec.getLowLim(),
      rec.getUpLim(),
      rec.getStep(),
      rec.geResult()
   });
}
}


/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
/* Create and display the form */
   java.awt.EventQueue.invokeLater(new Runnable() {
      public void run() {
         new NewJFrame().setVisible(true);
      }
   });
}
```

```
    // Variables declaration - do not modify
    private javax.swing.JButton AddColection;
    private javax.swing.JButton DeleteAll;
    private javax.swing.JButton jButtonAdd;
    private javax.swing.JButton jButtonDelete;
    private javax.swing.JButton jButtonResult;
    private javax.swing.JFrame jFrame1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JLabel jLabel4;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTable jTable1;
    private javax.swing.JTextField jTextFieldLow;
    private javax.swing.JTextField jTextFieldStep;
    private javax.swing.JTextField jTextFieldUp;
    // End of variables declaration
}
```

Класс RecIntegral:

```
package lab1.pkg1;

public class RecIntegral {

    private double lowLim;
    private double upLim;
    private double step;
    private double result;

    public RecIntegral(double lowLim, double upLim, double step) throws InvalidRec {
        if (!isValid(lowLim) || !isValid(upLim) || !isValid(step)) {
            throw new InvalidRec("Значения должны быть в диапазоне от 0.000001 до 1 000 000");
        }

        this.lowLim = lowLim;
        this.upLim = upLim;
        this.step = step;
        this.result = 0.0;
    }

    public RecIntegral(double lowLim, double upLim, double step, double result) throws InvalidRec {
        if (!isValid(lowLim) || !isValid(upLim) || !isValid(step)) {
            throw new InvalidRec("Значения должны быть в диапазоне от 0.000001 до 1 000 000");
        }

        this.lowLim = lowLim;
        this.upLim = upLim;
        this.step = step;
        this.result = result;
    }
```

```
    private boolean isValid(double value) {
        return value >= 0.000001 && value <= 1_000_000;
    }
    public double getLowLim() { return lowLim; }
    public double getUpLim() { return upLim; }
    public double getStep() { return step; }
    public double geResult() { return result; }

    public void setResult(double result) {
        this.result = result;
    }
    public double CalcIntegral(double lowLim, double upLim, double step) {
        double start = lowLim;
        double sumS = 0;

        do {
            double h = Math.min(step, (upLim - start));
            sumS += h * (1 / start + 1 / (start + h)) / 2;
            start += h;
        } while (start < upLim);

        return sumS;
    }
}
```

Класс InvalidRec:

```
package lab1.pkg1;

/**
 *
 * @author User
 */
```

```
public class InvalidRec extends Exception {
    public InvalidRec(String message) {
        super(message);
    }
}
```

РЕЗУЛЬТАТЫ ИСПЫТАНИЙ

Приложение Б

(обязательное)

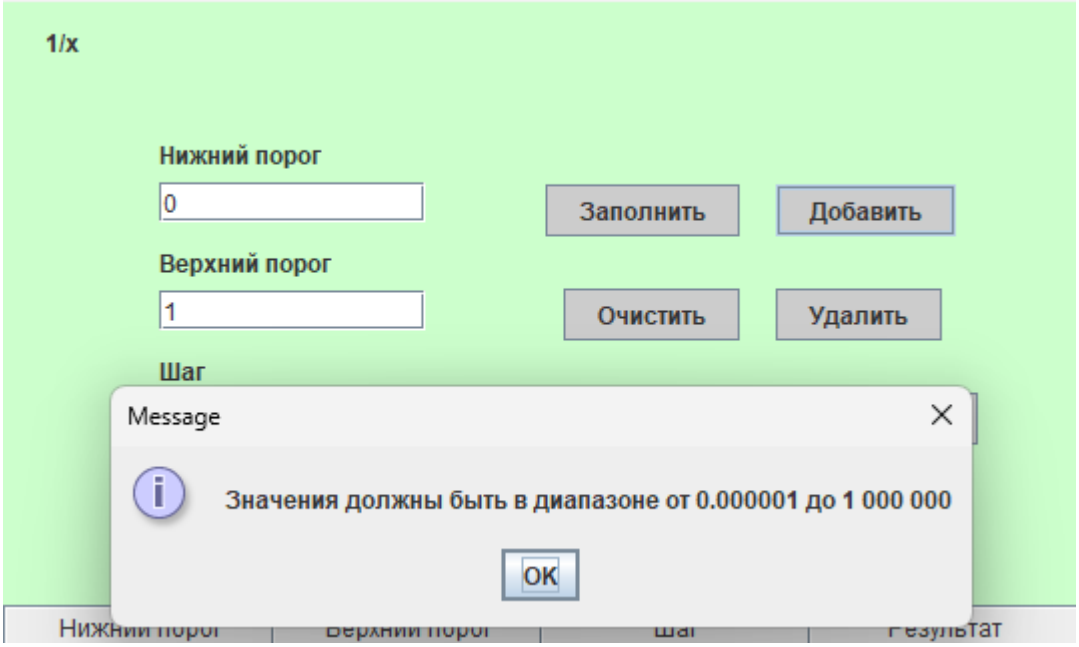

Рисунок Б.1 — Проверка корректности данных

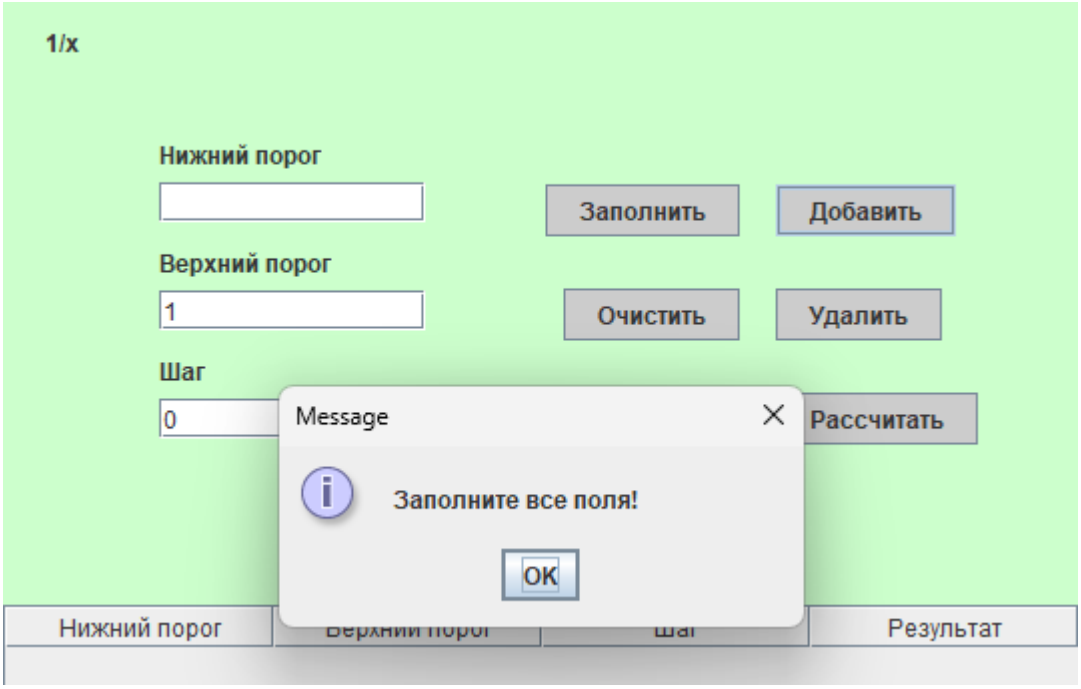

Рисунок Б.2 — Проверка ввода всех данных